



DoubleML

DoubleML

Multiple Treatment Levels and Treatment Effect Heterogeneity

DoubleML Workshop

February 27 & March 06, 2026

Economic AI

Multiple Treatment Levels

Motivation

- In many real-world settings, treatments are not just binary (treated vs. untreated), but can have **multiple levels** or represent **different types** of interventions.
- Estimating APOs allows us to understand the expected outcome for each specific treatment level or type, providing a richer picture than just comparing treated vs. untreated.
- Examples
 - Education Programs: Comparing no training, basic training, and advanced training (3 levels).
 - Medical Treatments: Patients receive different drugs (e.g., Drug A, Drug B, Drug C), not just “treatment” vs. “control”.
 - Marketing Strategies: Customers exposed to different types of campaigns (email, SMS, phone call), with no inherent ranking.
- *What would the average outcome be if everyone received a specific treatment level or type?*

Average Potential Outcomes (APOs)

- The **DoubleML** package provides a method to estimate the average potential outcomes (APOs)

$$\theta_d = \mathbb{E}[Y(d)]$$

for a given treatment assignment d (discrete treatment values)

- The corresponding score is given by

$$\psi(W; \theta, \eta) := g(d, X) + \frac{1\{D = d\}(Y - g(d, X))}{m_d(X)}$$

where the required regressions and classifications are

$$\begin{aligned} g(d, X) &= \mathbb{E}[Y | D = d, X] \\ m_d(X) &= \mathbb{E}[1\{D = d\} | X] \end{aligned}$$

Average Potential Outcomes (APOs)

- For a single treatment level d the DoubleMLAPO model

► Load data & Learners

```
1 print(f"Unique treatment values: {data['e401'].unique()}")
Unique treatment values: [0 1]

1 from doubleml import DoubleMLAPO
2
3 dml_apo = DoubleMLAPO(
4     dml_data,
5     ml_g=lgmb_reg,
6     ml_m=lgmb_clf,
7     treatment_level=1.0
8 )
9
10 _ = dml_apo.fit() # at this example we leave out tuning
11 dml_apo.summary.round(3)
```

	coef	std err	t	P> t	2.5 %	97.5 %
e401	21993.26	1076.635	20.428	0.0	19883.095	24103.425

Average Potential Outcomes (APOs)

- For multiple outcome levels DoubleMLAPoS model combines multiple DoubleMLAPo models

► Hyperparameter spaces & Settings

```
1 from doubleml import DoubleMLAPoS
2 dml_apos = DoubleMLAPoS(
3     dml_data,
4     ml_g=lgmb_reg,
5     ml_m=lgmb_clf,
6     treatment_levels=[0.0, 1.0]
7 )
8
9 dml_apos.tune_ml_models(
10     ml_param_space=param_space,
11     optuna_settings=optuna_settings,
12 )
13 _ = dml_apos.fit()
14 dml_apos.summary.round(3)
```

	coef	std err	t	P> t	2.5 %	97.5 %
0.0	13992.066	879.760	15.904	0.0	12267.768	15716.365
1.0	22284.840	939.222	23.727	0.0	20443.999	24125.681

- Allows for uniform/joint confidence intervals via bootstrap

Average Potential Outcomes (APOs)

- Further, DoubleMLAPoS models enable the estimation of differences in potential outcome levels via the `causal_contrast()` method

```
1 contrast_model = dml_apos.causal_contrast(reference_levels=[0.0])
2
3 contrast_model.summary.round(3)
```

	coef	std err	t	P> t	2.5 %	97.5 %
1.0 vs 0.0	8292.773	1203.207	6.892	0.0	5934.532	10651.015

- Compare the `causal_contrast()` to the outcome of a DoubleMLIRM model

► Fit DoubleMLIRM model

	coef	std err	t	P> t	2.5 %	97.5 %
e401	8087.427	1194.313	6.772	0.0	5746.616	10428.238

Treatment Effect Heterogeneity

Motivation

- In many cases, treatment effects are **heterogeneous** across individuals
- Think of the effect of a training program on wages:
 - The effect is likely to be different for individuals who participate in the program and those who do not
 - Usually, the goal is not to force everyone to do training programs
- Heterogeneity is of interest in many applications:
 - Effect of a marketing campaign on different types of customers (uplift modeling, e.g., different markets, different age groups, etc.)
 - Effect of a drug on different types of patients (e.g. age groups)

Group Average Treatment Effects

GATEs

- Often, we are interested in different subpopulations of individuals
- **Group Average Treatment Effects** (GATEs) are defined as the average treatment effect for a subpopulation of individuals (defined via an indicator G)

$$\tau_{GATE} = E[Y(1) - Y(0)|G = 1]$$

- Typical applications include:
 - Customer segmentation
 - Policy evaluation for different regions (e.g., urban vs. rural areas)
 - Assessing treatment effects by age group (e.g., young, middle-aged, elderly)

GATEs: General Idea

- The major idea to assess heterogeneous treatment effects is based on Semenova and Chernozhukov (2021)
- Consider the score for the IRM model for the ATE, which we can rewrite in terms of (Average) Potential Outcomes (APOs) (Knaus 2020)

$$\psi(W; \theta, \eta) := \left\{ \underbrace{g(1, X)}_{\rightsquigarrow \text{APO}(1)} + \frac{D(Y - g(1, X))}{\underbrace{m(X)}_{\rightsquigarrow \text{APO}(1)}} \right\} - \left\{ \underbrace{g(0, X)}_{\rightsquigarrow \text{APO}(0)} + \frac{(1 - D)(Y - g(0, X))}{\underbrace{1 - m(x)}_{\rightsquigarrow \text{APO}(0)}} \right\}$$

GATEs: General Idea

- The score for the ATE has a linear form, i.e., it can be rewritten as

$$\psi(W; \theta, \eta) = \psi_a(W; \eta)\theta + \psi_b(W; \eta)$$

- In the IRM, we have

$$\psi_a(W; \eta) = -1,$$

$$\psi_b(W; \eta) = g(1, X) + \frac{D(Y - g(1, X))}{m(X)} - g(0, X) - \frac{(1 - D)(Y - g(0, X))}{1 - m(x)}.$$

- The idea to estimate GATEs (and later CATEs) is to approximate the difference in the APOs, i.e., ψ_b
 - GATEs: Consider difference in APOs (i.e., ψ_b) for different groups
 - CATEs: Approximate difference in APOs (i.e., ψ_b) as a function of X

GATEs

- For DoubleMLIRM or DoubleMLPLR models, the GATEs can be estimated based on a standard model by using the `gate()` method
- Estimation of GATEs does not require re-estimation of the model. Instead we start with the previously fitted model

1 dml_irm.summary.round(3)

	coef	std err	t	P> t	2.5 %	97.5 %
e401	8087.427	1194.313	6.772	0.0	5746.616	10428.238

GATEs

- At first, we have to define groups and a corresponding pandas dataframe

```
1 import pandas as pd
2 groups = pd.DataFrame({'married': dml_data.data['marr']})
3 groups.head(n=3)
```

	married
0	0
1	0
2	0

- Then, we can call the `gate()` method and construct confidence intervals with the `confint()` method

```
1 import numpy as np
2 gate = dml_irm.gate(groups)
3 np.random.seed(123)
4 print(gate.confint(level=0.95, joint = True))
```

	2.5 %	effect	97.5 %
married	3489.61385	9157.212085	14824.810321

GATEs

- If we want to consider different groups, we could either call `gate()` multiple times or define the groups in a single dataframe

```
1 groups = pd.DataFrame({'married': dml_data.data['marr'] == 1,  
2                        'not_married': dml_data.data['marr'] == 0})  
3 groups.head(n=3)
```

	married	not_married
0	False	True
1	False	True
2	False	True

- This enables valid uniform confidence intervals

```
1 gate = dml_irm.gate(groups)  
2 np.random.seed(123)  
3 print(gate.confint(level=0.95, joint = True))
```

	2.5 %	effect	97.5 %
married	2922.165544	9157.212085	15392.258627
not_married	1518.268619	6449.983764	11381.698908

GATEs

- **Important:** For valid interpretation, the groups need to be **mutually exclusive!**
- A simple and intuitive way to set up mutually exclusive groups is to use only one column to define the different groups

```
1 marriage_status = dml_data.data['marr'].replace({1: 'married', 0: 'not married'})
2 groups = pd.DataFrame({'marriage_status': marriage_status})
3 groups.head(n=3)
```

marriage_status	
0	not married
1	not married
2	not married

```
1 gate = dml_irm.gate(groups)
2 print(gate.confint(level=0.95))
```

	2.5 %	effect	97.5 %
Group_married	5719.236442	9157.212085	12595.187729
Group_not married	3730.658987	6449.983764	9169.308541

Conditional Average Treatment Effects

CATEs

- **Conditional Average Treatment Effects (CATEs)** are defined as the average treatment effect for a subpopulation of individuals (defined via a vector of covariates X)

$$\tau(x) = E[Y(1) - Y(0)|X = x]$$

- Depending on X this can be quite complicated to estimate (e.g. if $X = (X_1, X_2, X_3, \dots)$ includes multiple variables)
- A very simple idea is to approximate $\tau(x)$ as a linear function of X (see Semenova and Chernozhukov (2021)):

$$\tau(x) \approx \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

- The coefficients only describe the heterogeneity in the treatment effect of D on Y with respect to X . This does not mean X affects D or D affects X !

Basic CATEs

- Consider the conditional average effect of **age**

```
1 age_data = dml_data.data["age"]
2 linear_basis = pd.DataFrame({'intercept': np.ones_like(age_data),
3                               'age': age_data})
4 cate_linear = dml_irm.cate(linear_basis)
5 print(cate_linear)
```

===== DoubleMLBLP Object =====

----- Fit summary -----

	coef	std err	t	P> t	[0.025	0.975]
intercept	-4228.386210	4995.288945	-0.846475	0.397288	-14018.972634	5562.200214
age	299.945191	137.514855	2.181184	0.029170	30.421027	569.469354

Basic CATEs

- Usually, we are not interested in the coefficients (or confidence intervals), but confidence values for new observations

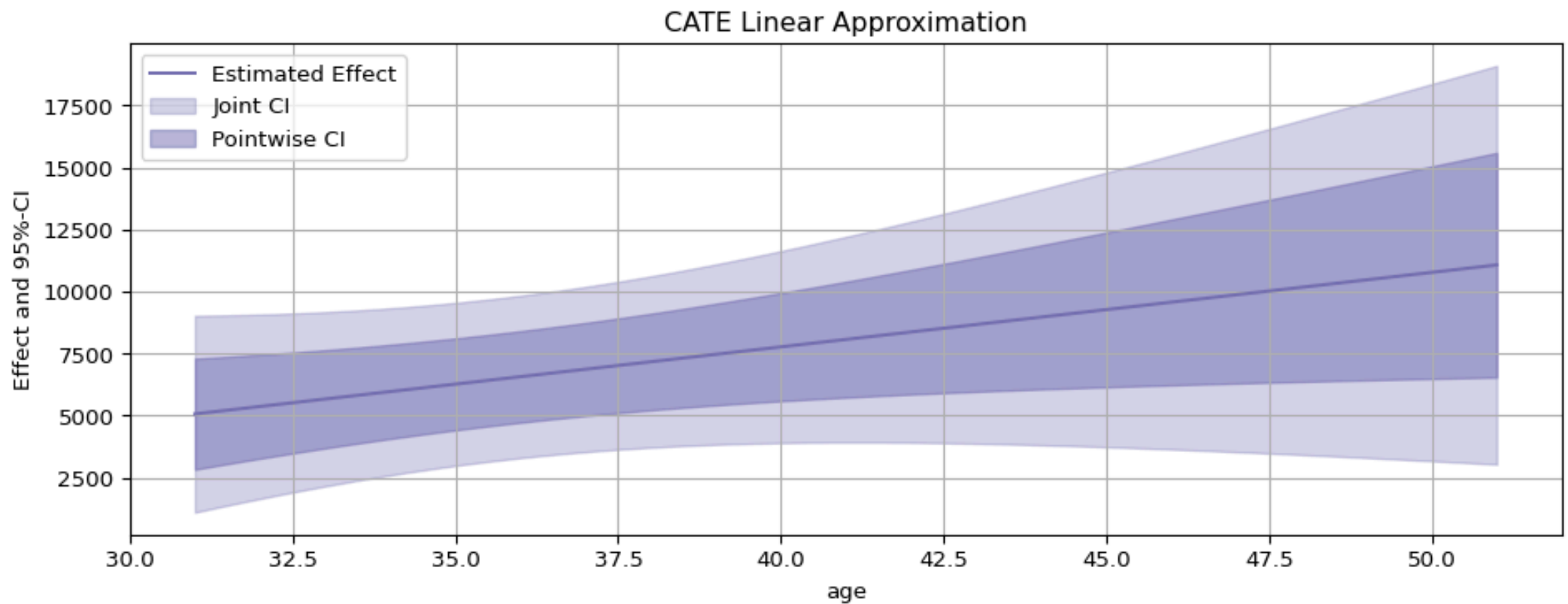
```
1 new_data = {"age": np.linspace(np.quantile(age_data, 0.2), np.quantile(age_data, 0.8), 50)}  
2 linear_grid = pd.DataFrame({"intercept": np.ones_like(new_data["age"]),  
3                             "age": new_data["age"]})  
4 df_cate_linear = cate_linear.confint(linear_grid, level=0.95, joint=True, n_rep_boot=2000)  
5 print(df_cate_linear.head(n=8))
```

	2.5 %	effect	97.5 %
0	1114.091612	5069.914706	9025.737801
1	1343.726823	5192.341315	9040.955806
2	1566.245411	5314.767923	9063.290435
3	1781.062845	5437.194532	9093.326219
4	1987.581504	5559.621140	9131.660777
5	2185.202630	5682.047749	9178.892868
6	2373.341196	5804.474357	9235.607519
7	2551.443348	5926.900966	9302.358583

Basic CATEs

- This can be easily visualized

► Code



Polynomial CATE Approximations

- Since the linear approximation is quite restrictive, we can also use a more flexible approximation, e.g. polynomial

$$\tau(x) \approx \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \dots$$

```
1 from sklearn.preprocessing import PolynomialFeatures
2
3 # Create the polynomial features object
4 poly = PolynomialFeatures(degree=3)
5 poly_basis_array = poly.fit_transform(dml_data.data[["age"]])
6 poly_basis = pd.DataFrame(poly_basis_array, columns=poly.get_feature_names_out())
7 print(poly_basis.head())
```

	1	age	age^2	age^3
0	1.0	47.0	2209.0	103823.0
1	1.0	36.0	1296.0	46656.0
2	1.0	37.0	1369.0	50653.0
3	1.0	58.0	3364.0	195112.0
4	1.0	32.0	1024.0	32768.0

Polynomial CATE Approximations

- Now, we can estimate the CATEs based on the polynomial basis

```
1 cate_poly = dml_irm.cate(poly_basis)
2 print(cate_poly)
```

```
===== DoubleMLBLP Object =====

----- Fit summary -----
```

	coef	std err	t	P> t	[0.025	0.975]
1	-74888.543014	76219.777227	-0.982534	0.325837	-224276.561290	74499.475261
age	5077.958840	5864.049073	0.865948	0.386519	-6415.366146	16571.283826
age^2	-102.098702	144.840852	-0.704903	0.480871	-385.981557	181.784152
age^3	0.694393	1.153037	0.602230	0.547021	-1.565518	2.954304

Polynomial CATE Approximations

- Create a regular grid of new observations

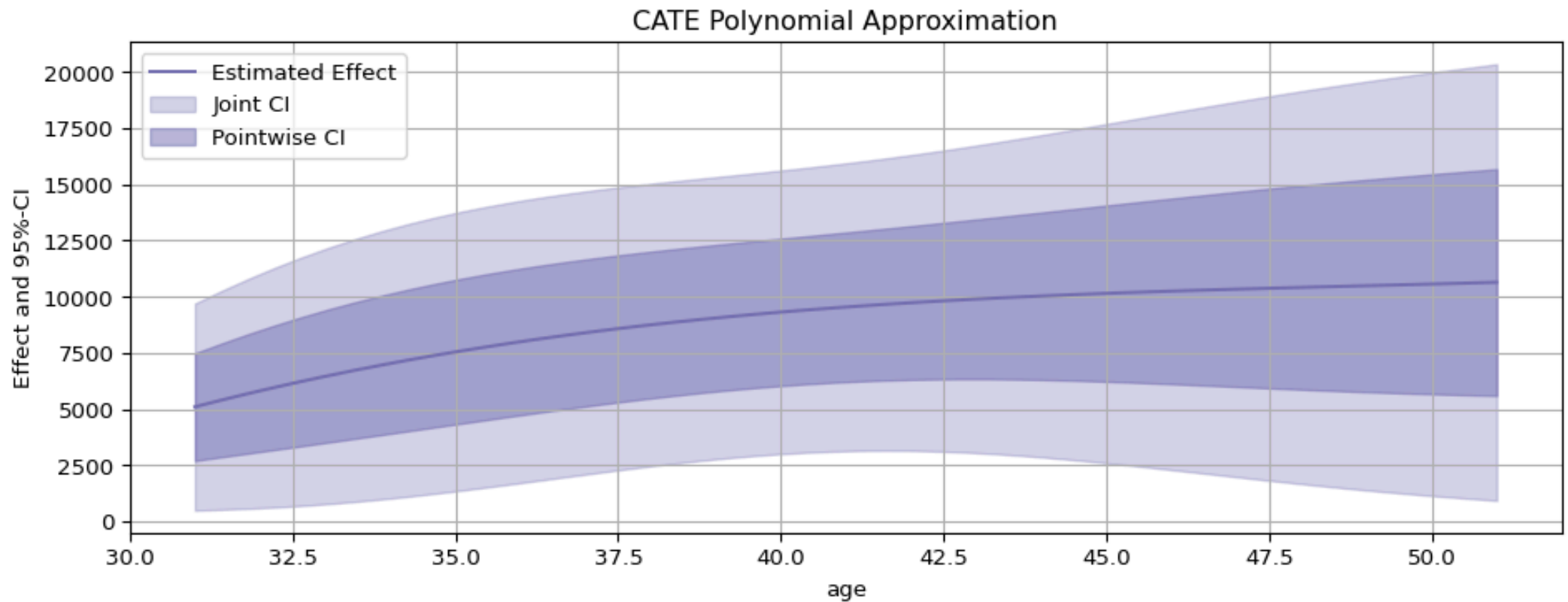
```
1 poly_grid = pd.DataFrame(poly.transform(new_data["age"].reshape(-1,1)),  
2                           columns=poly.get_feature_names_out())  
3 df_cate_poly = cate_poly.confint(poly_grid, level=0.95, joint=True, n_rep_boot=2000)  
4 print(df_cate_poly.head(n=8))
```

	2.5 %	effect	97.5 %
0	496.372650	5097.993624	9699.614597
1	533.041764	5397.820569	10262.599374
2	577.754021	5685.429317	10793.104613
3	633.732173	5961.103175	11288.474178
4	702.661034	6225.125452	11747.589871
5	785.173050	6477.779455	12170.385861
6	881.161803	6719.348492	12557.535181
7	989.986892	6950.115870	12910.244848

Polynomial CATE Approximations

- And visualize the results

► Code



Spline CATE Approximations

- Splines offer a more flexible approximation than polynomials
- B-splines are piecewise polynomial functions $B_j(x)$ that are smooth at the knots

$$\tau(x) \approx \sum_{j=1}^k \beta_j B_j(x)$$

Spline CATE Approximations

- Now, we can estimate the CATEs based on the spline basis

```
1 cate_spline = dml_irm.cate(spline_basis)
2 print(cate_spline)
```

```
===== DoubleMLBLP Object =====

----- Fit summary -----
      coef      std err      t      P>|t|      [0.025      0.975]
spline_0  29893.449825  44771.837827  0.667684  0.504335 -57857.739837  117644.639488
spline_1  -7950.323272   9747.556993 -0.815622  0.414716 -27055.183915  11154.537371
spline_2  12800.554686   6406.046344  1.998199  0.045695   244.934569  25356.174804
spline_3   5183.859088   6793.097883  0.763107  0.445400 -8130.368105  18498.086281
spline_4  22342.816025  10979.348238  2.034986  0.041852   823.688906  43861.943144
spline_5 -21032.986339  29974.315658 -0.701700  0.482866 -79781.565491  37715.592813
spline_6  226409.181433 156155.747507  1.449893  0.147088 -79650.459659  532468.822525
```

Spline CATE Approximations

- Create a regular grid of new observations

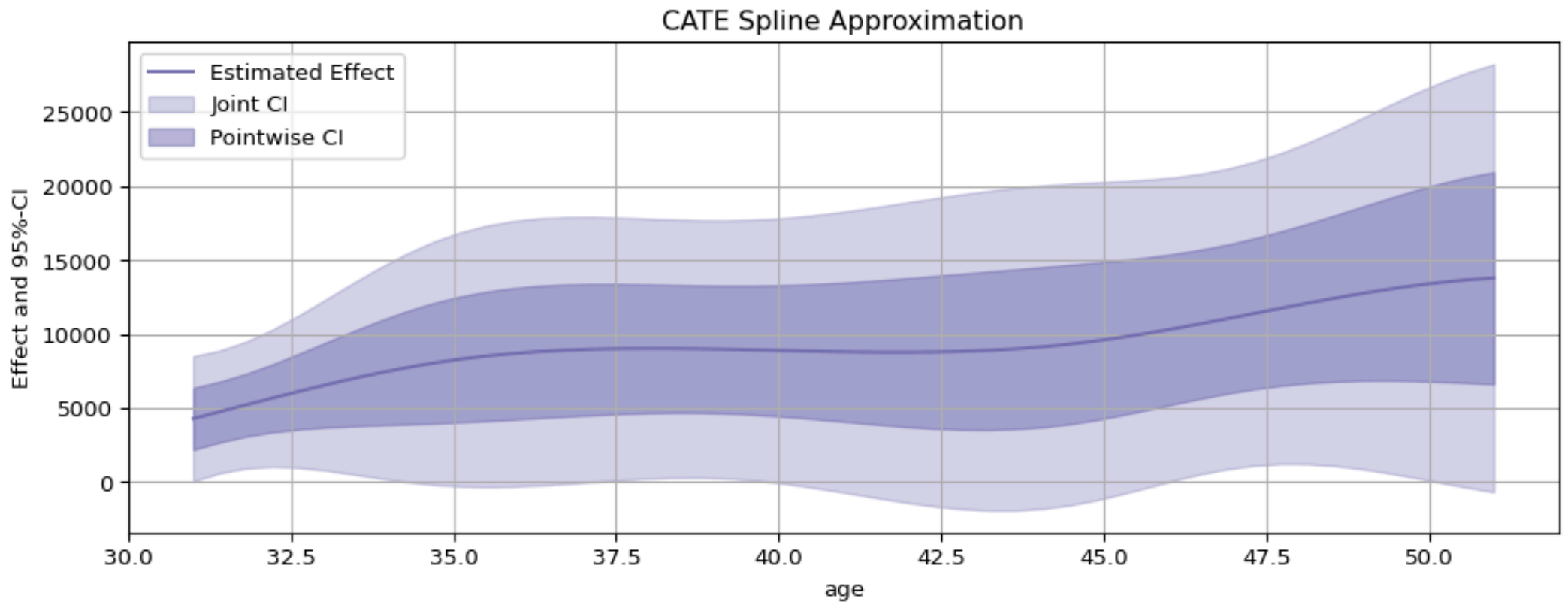
```
1 spline_grid = pd.DataFrame(spline.transform(new_data["age"].reshape(-1,1)),  
2                             columns=[f"spline_{i}" for i in range(spline_basis_array.shape[1])])  
3 df_cate_spline = cate_spline.confint(spline_grid, level=0.95, joint=True, n_rep_boot=2000)  
4 print(df_cate_spline.head(n=8))
```

	2.5 %	effect	97.5 %
0	50.955507	4273.258561	8495.561614
1	596.206344	4739.046060	8881.885777
2	923.455875	5207.355314	9491.254754
3	1029.209969	5671.806355	10314.402740
4	956.612560	6126.019212	11295.425865
5	766.597371	6563.613918	12360.630466
6	516.458758	6978.210505	13439.962252
7	253.073971	7363.429003	14473.784035

Spline CATE Approximations

- And visualize the results

► Code



Conditional Average Potential Outcomes

CAPOs

- **C**onditional **A**verage **P**otential **O**utcomes (CAPOs) extend the APO concept to allow for heterogeneity

$$\theta_d(x) = \mathbb{E}[Y(d)|X = x]$$

- CAPOs describe how the expected potential outcome under treatment level d varies with covariates X
- Similar to CATEs, we can approximate CAPOs using basis functions:

$$\theta_d(x) \approx \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

- The **DoubleMLAPO** model provides the **capo()** method for estimation

CAPOs: Setup

- First, we reuse the DoubleMLAP0S model with multiple treatment levels

1 dml_apos.summary.round(3)

	coef	std err	t	P> t	2.5 %	97.5 %
0.0	13992.066	879.760	15.904	0.0	12267.768	15716.365
1.0	22284.840	939.222	23.727	0.0	20443.999	24125.681

- The model includes two DoubleMLAP0 models in the modellist attribute

1 print(dml_apos.treatment_levels)
2 dml_apo_lvl0 = dml_apos.modellist[0]
3 dml_apo_lvl1 = dml_apos.modellist[1]

[0.0, 1.0]

CAPOs: Linear Approximation

- Estimate CAPOs with a linear basis for **age**

```
1 age_data = dml_data.data["age"]
2 linear_basis_capo = pd.DataFrame({'intercept': np.ones_like(age_data),
3                                   'age': age_data})
4 capo_lvl0_linear = dml_apo_lvl0.capo(linear_basis_capo)
5 capo_lvl1_linear = dml_apo_lvl1.capo(linear_basis_capo)
6 print(f"CAPO (Level 0):\n{capo_lvl0_linear}\n")
7 print(f"CAPO (Level 1):\n{capo_lvl1_linear}")
```

CAPO (Level 0):

===== DoubleMLBLP Object =====

----- Fit summary -----

	coef	std err	t	P> t	[0.025	0.975]
intercept	-30282.618381	3576.450741	-8.467226	2.513052e-17	-37292.333026	-23272.903736
age	1078.286812	98.137572	10.987502	4.389035e-28	885.940705	1270.632918

CAPO (Level 1):

===== DoubleMLBLP Object =====

----- Fit summary -----

	coef	std err	t	P> t	[0.025	0.975]
intercept	-35558.229659	3863.676262	-9.203211	3.474083e-20	-43130.895981	-27985.563338
age	1408.737732	106.807518	13.189500	1.008522e-39	1199.398844	1618.076620

CAPOs: Linear Approximation

- Create confidence intervals for new observations

```
1 new_data_capo = {"age": np.linspace(np.quantile(age_data, 0.2), np.quantile(age_data, 0.8), 50)}
2 linear_grid_capo = pd.DataFrame({"intercept": np.ones_like(new_data_capo["age"]),
3                                  "age": new_data_capo["age"]})
4 df_capo_lvl0_linear = capo_lvl0_linear.confint(linear_grid_capo, level=0.95, joint=True, n_rep_boot=2000)
5 df_capo_lvl1_linear = capo_lvl1_linear.confint(linear_grid_capo, level=0.95, joint=True, n_rep_boot=2000)
6 print(f"CAPO (Level 0):\n{df_capo_lvl0_linear.head(n=5)}\n")
7 print(f"CAPO (Level 1):\n{df_capo_lvl1_linear.head(n=5)}")
```

CAPO (Level 0):

	2.5 %	effect	97.5 %
0	-362.936927	3144.272781	6651.482488
1	167.006960	3584.389847	7001.772733
2	690.970874	4024.506913	7358.042951
3	1208.492823	4464.623979	7720.755135
4	1719.103188	4904.741045	8090.378901

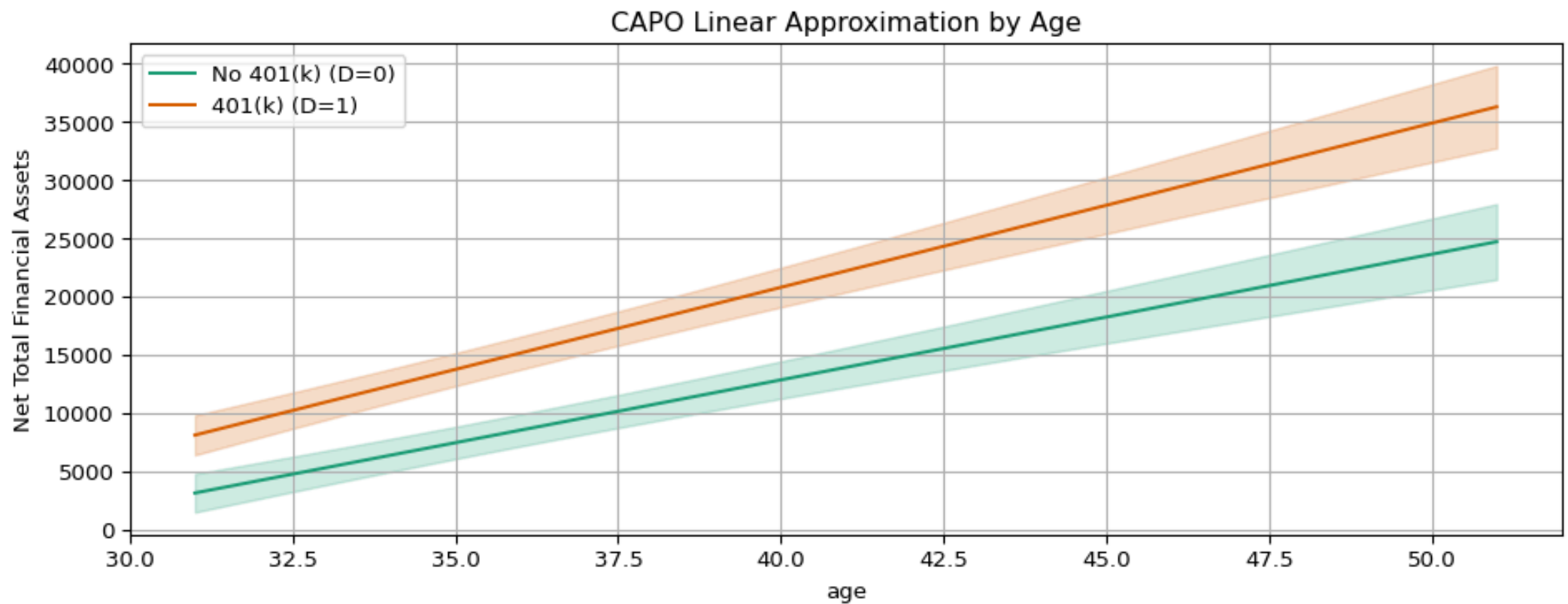
CAPO (Level 1):

	2.5 %	effect	97.5 %
0	4053.352047	8112.640030	12171.928014
1	4739.344027	8687.635023	12635.926019
2	5417.667655	9262.630016	13107.592377
3	6087.688976	9837.625008	13587.561041
4	6748.761944	10412.620001	14076.478058

CAPOs: Linear Approximation

- Visualize the conditional average potential outcomes for both treatment levels

► Code



CAPOs: Spline Approximation

- Use spline basis for more flexible approximation

```
1 from sklearn.preprocessing import SplineTransformer
2
3 spline_capo = SplineTransformer(n_knots=5, degree=3, include_bias=True)
4 spline_basis_capo = pd.DataFrame(
5     spline_capo.fit_transform(dml_data.data[["age"]]),
6     columns=[f"spline_{i}" for i in range(spline_capo.n_features_out_)]
7 )
8
9 capo_lvl0_spline = dml_apo_lvl0.capo(spline_basis_capo)
10 capo_lvl1_spline = dml_apo_lvl1.capo(spline_basis_capo)
11 print(f"CAPO (Level 0):\n{capo_lvl0_spline}")
```

CAPO (Level 0):

===== DoubleMLBLP Object =====

----- Fit summary -----

	coef	std err	t	P> t	[0.025	0.975]
spline_0	14770.662352	28418.522191	0.519755	0.603235	-40928.617636	70469.942340
spline_1	-2092.818310	6423.800274	-0.325791	0.744582	-14683.235492	10497.598871
spline_2	4818.673875	4235.079683	1.137800	0.255204	-3481.929776	13119.277526
spline_3	20195.525403	4216.931188	4.789152	0.000002	11930.492149	28460.558657
spline_4	15567.378250	6814.043913	2.284602	0.022336	2212.097591	28922.658908
spline_5	75394.099923	22824.519100	3.303207	0.000956	30658.864522	120129.335324
spline_6	-115412.303799	142447.037473	-0.810212	0.417818	-394603.366950	163778.759353

CAPOs: Spline Approximation

- Create confidence intervals on a grid

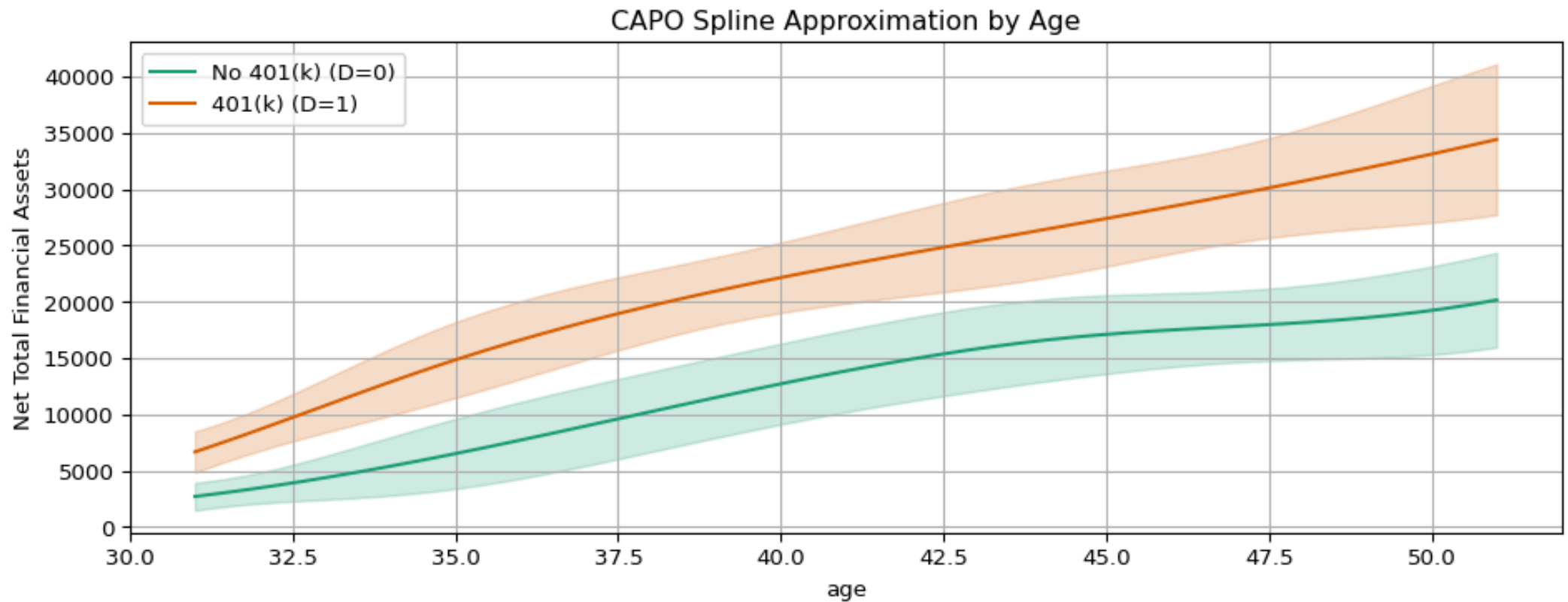
```
1 spline_grid_capo = pd.DataFrame(  
2     spline_capo.transform(new_data_capo["age"].reshape(-1, 1)),  
3     columns=[f"spline_{i}" for i in range(spline_capo.n_features_out_)]  
4 )  
5 df_capo_lvl0_spline = capo_lvl0_spline.confint(spline_grid_capo, level=0.95, joint=True, n_rep_boot=2000)  
6 df_capo_lvl1_spline = capo_lvl1_spline.confint(spline_grid_capo, level=0.95, joint=True, n_rep_boot=2000)  
7 print(df_capo_lvl0_spline.head(n=5))
```

	2.5 %	effect	97.5 %
0	200.522493	2714.658567	5228.794641
1	513.031702	3009.870206	5506.708710
2	655.696022	3329.113502	6002.530983
3	646.307326	3671.265267	6696.223209
4	537.238795	4035.202315	7533.165835

CAPOs: Spline Approximation

- Visualize the spline-based CAPOs

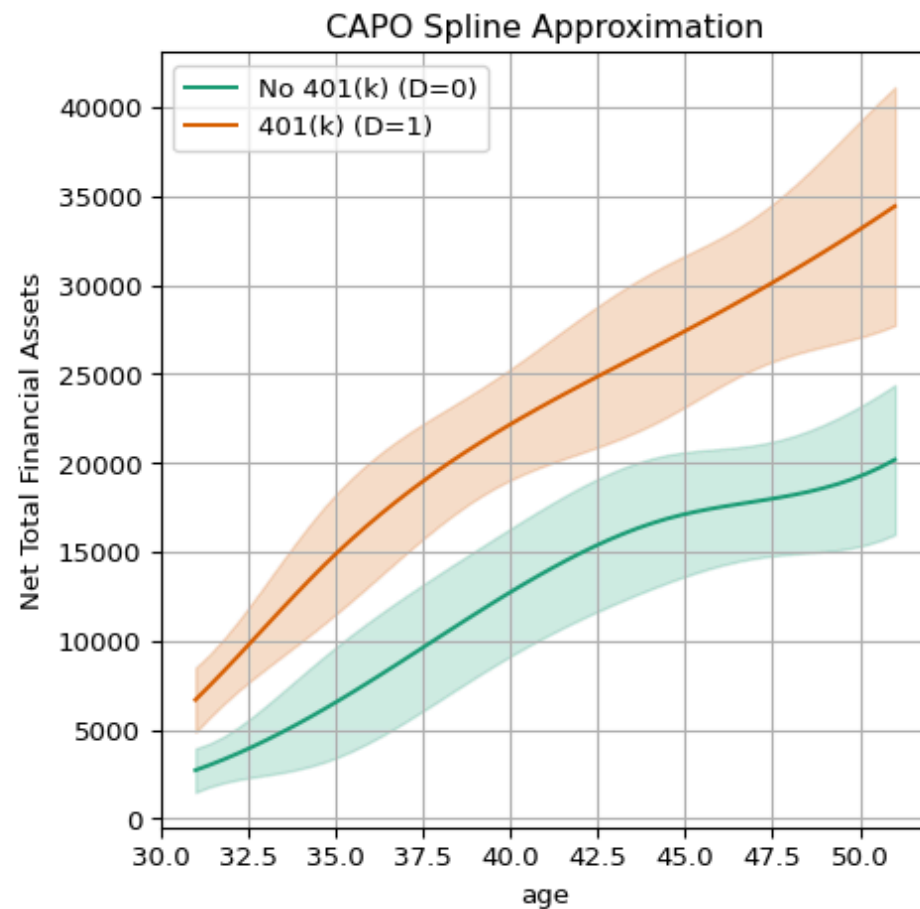
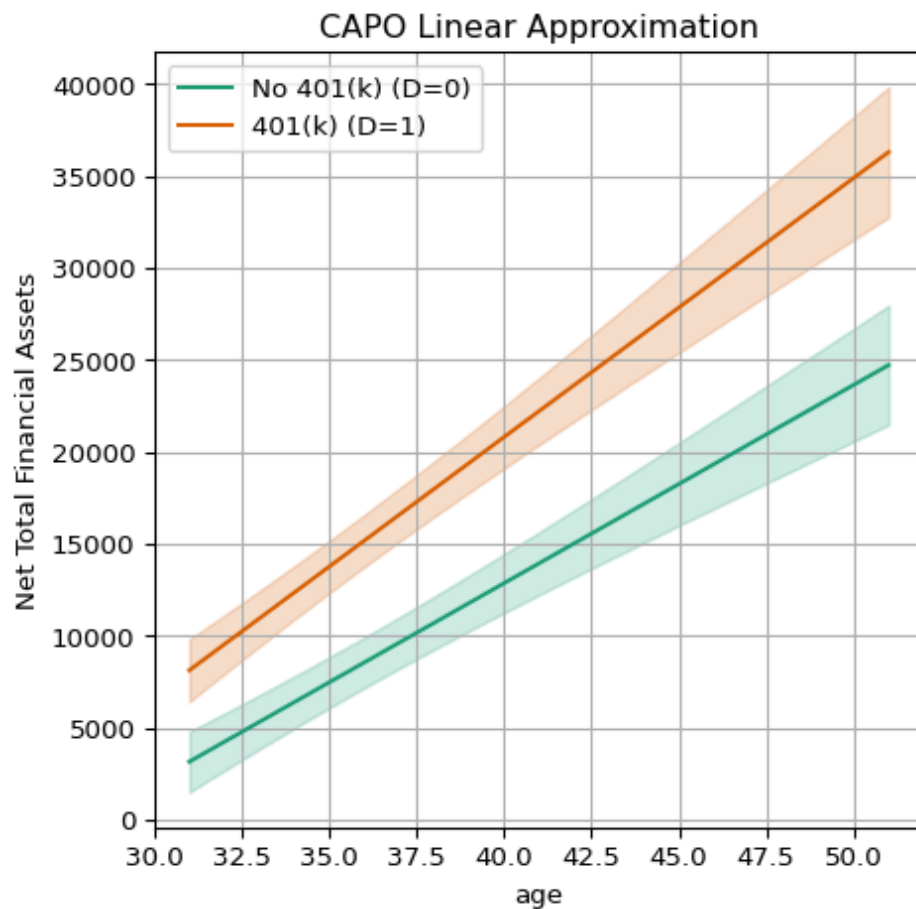
► Code



Comparing CAPOs: Treatment Effect Heterogeneity

- We can compare the CAPOs to visualize treatment effect heterogeneity across age

► Code



CAPOs vs CATEs

- **CATEs** estimate the treatment effect heterogeneity: $\tau(x) = \mathbb{E}[Y(1) - Y(0)|X = x]$
- **CAPOs** estimate the potential outcome heterogeneity: $\theta_d(x) = \mathbb{E}[Y(d)|X = x]$
- Key differences:
 - CATEs directly give you the effect of treatment
 - CAPOs give you the expected outcome under each treatment level
 - CAPOs are useful when you want to understand the outcome levels, not just differences
 - Multiple treatment levels: CAPOs naturally extend to > 2 treatment levels
- The difference between CAPOs gives the CATE:

$$\tau(x) = \theta_1(x) - \theta_0(x)$$

For a more detailed comparison, see this [Example Notebook](#)

Outlook: Further Topics

Quantile Treatment Effects (QTEs)

- QTEs (from Kallus, Mao, and Uehara (2019)) are implemented via a separate class

```
1 from doubleml import DoubleMLQTE
2 from lightgbm import LGBMClassifier, LGBMRegressor
3 from sklearn.base import clone
4
5 tau_vec = np.arange(0.1, 0.95, 0.1)
6 n_folds = 5
7
8 # Learners
9 class_learner = LGBMClassifier(n_estimators=300, learning_rate=0.05, num_leaves=10, verbose=-1)
10
11 np.random.seed(42)
12 dml_QTE = DoubleMLQTE(
13     dml_data, ml_g=clone(class_learner), ml_m=clone(class_learner), quantiles=tau_vec, score="PQ", normalize
14 )
15 _ = dml_QTE.fit()
16 print(dml_QTE)
```

===== DoubleMLQTE Object =====

----- Fit summary -----						
	coef	std err	t	P> t	2.5 %	97.5 %
0.1	1210.0	486.438569	2.487467	1.286563e-02	256.597923	2163.402077
0.2	1211.0	252.052811	4.804549	1.551010e-06	716.985569	1705.014431
0.3	622.0	255.252133	2.436806	1.481761e-02	121.715013	1122.284987
0.4	2006.0	320.163566	6.265547	3.715179e-10	1378.490941	2633.509059
0.5	4601.0	448.109454	10.267581	0.000000e+00	3722.721609	5479.278391
0.6	7040.0	605.739720	11.622153	0.000000e+00	5852.771965	8227.228035
0.7	10928.0	859.705581	12.711328	0.000000e+00	9243.008023	12612.991977

Quantile Treatment Effects (QTEs)

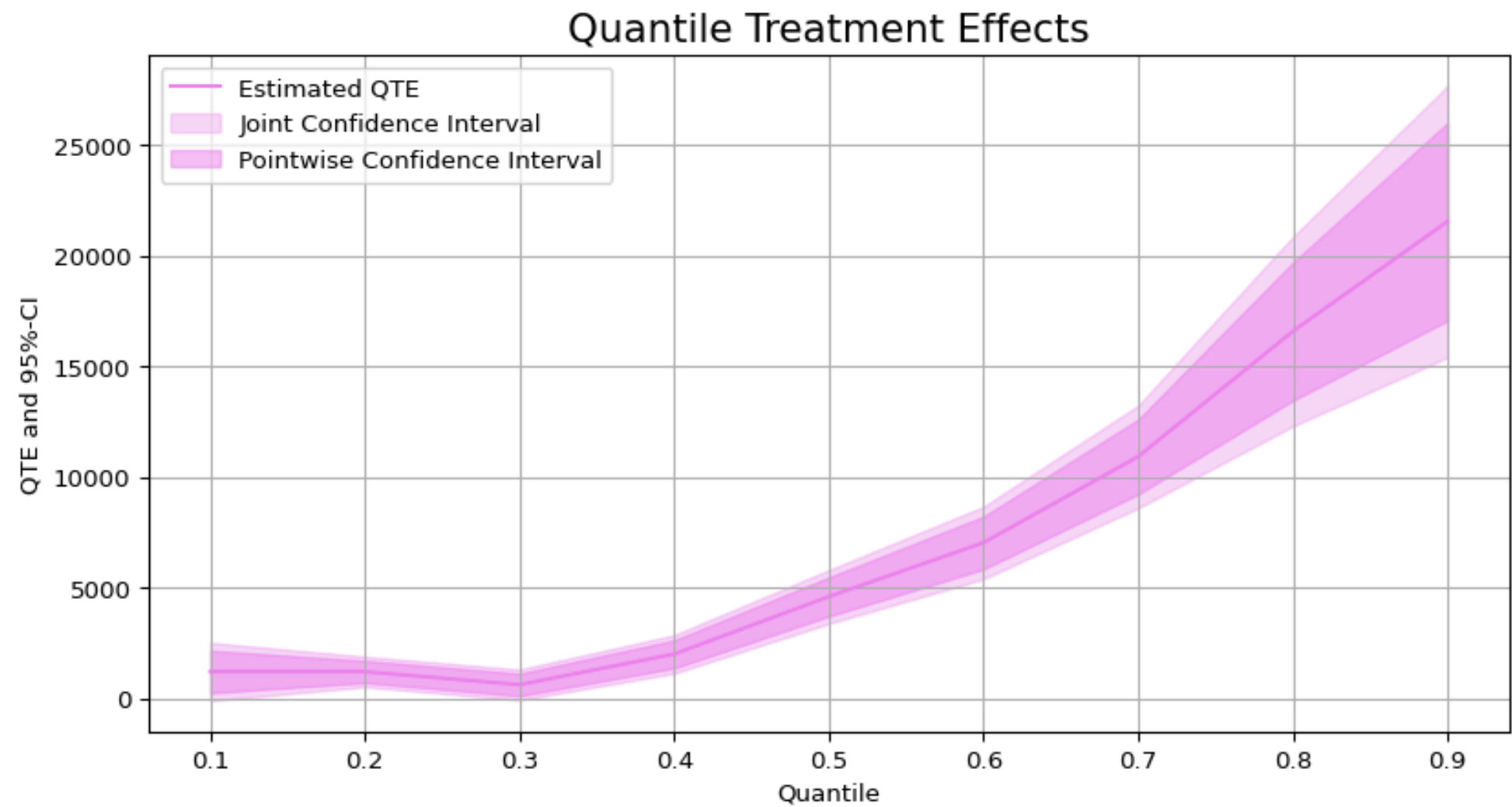
- Create simultaneously valid confidence intervals

```
1 _ = dml_QTE.bootstrap(n_rep_boot=2000)
2 ci_QTE = dml_QTE.confint(level=0.95, joint=True)
3
4 print(ci_QTE)
```

	2.5 %	97.5 %
0.1	-97.964554	2517.964554
0.2	533.265620	1888.734380
0.3	-64.336904	1308.336904
0.4	1145.125426	2866.874574
0.5	3396.097019	5805.902981
0.6	5411.251515	8668.748485
0.7	8616.373212	13239.626788
0.8	12316.337390	20863.662610
0.9	15421.942072	27678.057928

Quantile Treatment Effects (QTEs)

► Code

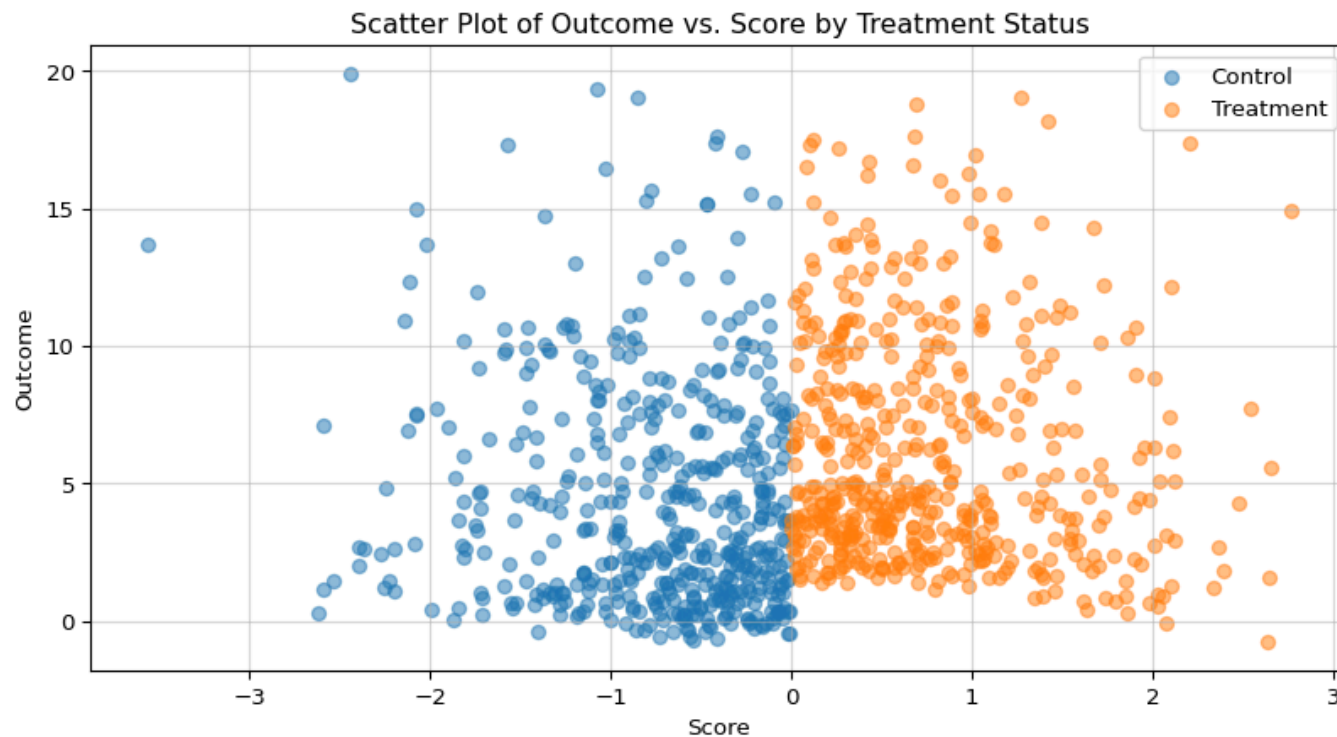


Regression Discontinuity Designs (RDD)

- Parameter of interest is the **Local Average Treatment Effect** (LATE) at some cutoff value point c

$$\theta_0 = \mathbb{E}[Y(1) - Y(0) | S = c]$$

► Code



Regression Discontinuity Designs (RDD)

- Flexible covariate adjustment as in Noack, Olma, and Rothe (2024)
- Support fuzzy RDDs and global learner versions (see documentation)

```
1 from doubleml.data import DoubleMLRDDData
2 from doubleml.rdd import RDFlex
3
4 dml_data_sharp = DoubleMLRDDData(df, y_col='y', d_cols='d', x_cols=cov_names, score_col='score')
5 ml_g = LGBMRegressor(n_estimators=500, learning_rate=0.01, verbose=-1)
6
7 rdflex_sharp = RDFlex(dml_data_sharp,
8                       ml_g,
9                       cutoff=0,
10                      fuzzy=False,
11                      n_folds=5,
12                      n_rep=1)
13 rdflex_sharp.fit(n_iterations=2)
14
15 print(rdflex_sharp)
```

Method	Coef.	S.E.	t-stat	P> t	95% CI
Conventional	2.014	0.103	19.494	1.231e-84	[1.812, 2.217]
Robust	–	–	16.872	7.173e-64	[1.779, 2.247]
Design Type:	Sharp				
Cutoff:	0				
First Stage Kernel:	triangular				
Final Bandwidth:	[0.6607402]				

Policy Learning with Trees

- The **DoubleML** package also supports basic policy learning with trees for the **DoubleMLIRM** model (similar to Athey and Wager (2021))
- The idea is to use a classification tree to optimize over feature regions
- Let X be the set of covariates for which we want to optimize the treatment assignment with a policy

$$\pi : X \rightarrow \{0, 1\}$$

- The policy is then defined by maximizing the following objective function

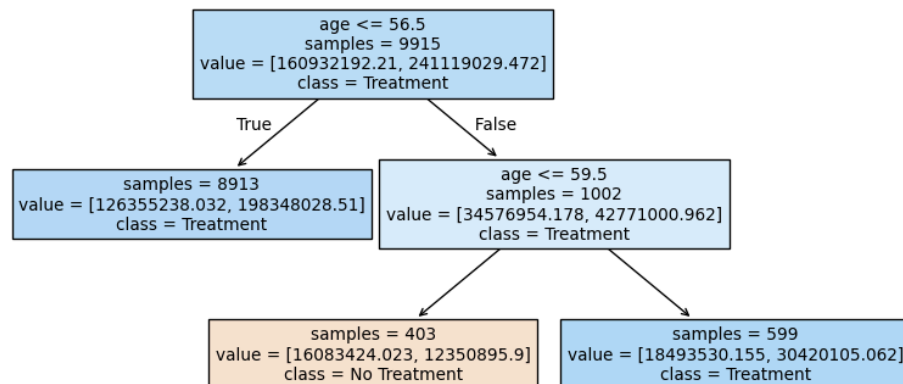
$$\frac{1}{n} \sum_{i=1}^n \underbrace{(2\pi(X_i) - 1)}_{\text{policy decision}} \underbrace{\psi_b(W_i, \hat{\eta})}_{\text{effect size}} = \frac{1}{n} \sum_{i=1}^n (2\pi(X_i) - 1) \underbrace{\text{sign}(\psi_b(W_i, \hat{\eta}))}_{\text{label}} \underbrace{|\psi_b(W_i, \hat{\eta})|}_{\text{weight}}.$$

Policy Learning with Trees

- Policy trees can be fitted via the `policy_tree()` method
- As the tree is based on the scores, the learners do not have to be re-estimated

```
1 features = dml_data.data[["age", "marr"]]
2 policy_tree = dml_irm.policy_tree(features=features, ccp_alpha = 0.001)
3
4 policy_tree.plot_tree()
```

```
[Text(0.4, 0.8333333333333334, 'age <= 56.5\nsamples = 9915\nvalue = [160932192.21, 241119029.472]\nclass = Treatment'),
Text(0.2, 0.5, 'samples = 8913\nvalue = [126355238.032, 198348028.51]\nclass = Treatment'),
Text(0.30000000000000004, 0.6666666666666667, 'True '),
Text(0.6, 0.5, 'age <= 59.5\nsamples = 1002\nvalue = [34576954.178, 42771000.962]\nclass = Treatment'),
Text(0.5, 0.6666666666666667, ' False'),
Text(0.4, 0.16666666666666666, 'samples = 403\nvalue = [16083424.023, 12350895.9]\nclass = No Treatment'),
Text(0.8, 0.16666666666666666, 'samples = 599\nvalue = [18493530.155, 30420105.062]\nclass = Treatment')]
```



Policy Learning with Trees

Recommendation

- Evaluation of learned policies should be performed on a **separate** test set
- Shallow trees are recommended for policy learning
- Policy tree implementation is an approximation; a formal framework is provided in Athey and Wager (2021)

Copyright



Economic AI GmbH
Nürnberger Str. 262 A
93059 Regensburg
Germany

info@economicaai.com

<https://www.economicaai.com>

Copyright ©2024 Economic AI GmbH. No part of this publication may be reproduced, stored in a retrieval system, used in a spreadsheet or document, or transmitted in any form or by any means electronic, mechanical, photocopying, recording, or otherwise without the permission of Economic AI.

References

References

- Athey, Susan, and Stefan Wager. 2021. “Policy Learning with Observational Data.” *Econometrica* 89 (1): 133–61.
- Bach, Philipp, Victor Chernozhukov, Malte S Kurz, and Martin Spindler. 2022. “DoubleML-an Object-Oriented Implementation of Double Machine Learning in Python.” *Journal of Machine Learning Research* 23: 53–51.
- Bach, Philipp, Malte S. Kurz, Victor Chernozhukov, Martin Spindler, and Sven Klaassen. 2024. “DoubleML: An Object-Oriented Implementation of Double Machine Learning in R.” *Journal of Statistical Software*. <https://doi.org/10.18637/jss.v108.i03>.
- Chernozhukov, Victor, Christian Hansen, Nathan Kallus, Martin Spindler, and Vasilis Syrgkanis. 2024. *Applied Causal Inference Powered by Machine Learning and AI*. CausalML-book.org. <https://causalml-book.org>.
- Kallus, Nathan, Xiaojie Mao, and Masatoshi Uehara. 2019. “Localized Debiased Machine Learning: Efficient Inference on Quantile Treatment Effects and Beyond.” *arXiv Preprint arXiv:1912.12945*.
- Knaus, Michael C. 2020. “Double Machine Learning Based Program Evaluation Under Unconfoundedness.” *arXiv Preprint arXiv:2003.03191*. <https://github.com/MCKnaus/causalDML>.
- Noack, Claudia, Tomasz Olma, and Christoph Rothe. 2024. “Flexible Covariate Adjustments in Regression Discontinuity Designs.” <https://arxiv.org/abs/2107.07942>.
- Semenova, Vira, and Victor Chernozhukov. 2021. “Debiased Machine Learning of Conditional Average Treatment Effects and Other Causal Functions.” *The Econometrics Journal* 24 (2): 264–89.